A Configuration-Driven Architecture for Multi-Scene Industrial-Agent Integration

***Abstract*—Industrial agents must operate across scenes with different signals, protocols, and limits. Yet a gap remains in connecting replaceable agent runtimes to current process context and governed device operations. This paper presents a configuration-driven architecture that separates harness execution from named operations, binds semantic acquisition and control nodes to protocol drivers and scene guards, and supplies the active line–product–recipe–run context at each turn. Shared dispatch checks, route-specific admission limits, conditional approval, and context revalidation govern writes; device receipts remain separate from subsequent process observations. Evaluation combined a five-protocol simulator matrix and scene-onboarding replay with one agent-mediated thermal-twin trial, six registered-harness tasks, mock approval tests, four direct-node missions, and a deterministic paired-entry check. Experimental results show that 4/4 writes in the paired two-scene, two-route check had verified 1/1 receipts and matching device readbacks. In the thermal trial, independent reads confirmed all three setpoints, and the agent retrieved a later sample matching the platform record. Six task probes remained below their target bands; each of four one-trajectory missions reported one endpoint within its band. The architecture provides a software integration layer for the tested routes and scenes, but the evidence does not establish multi-agent performance, repeatability, process benefit, or physical-plant performance.**

***Index Terms*—Industrial agents, multi-agent systems, industrial automation, production context, industrial communication.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect a changing process context to heterogeneous equipment without giving a language model unrestricted access to control points. A line-speed proposal, for example, depends on the active product, recipe, run, engineering limits, and node that represents the control. It must also reach the correct device through a configured protocol, and its receipt must not be confused with a later process response. When these elements are implemented separately for each agent and production scene, integrating a new engine or line requires repeating substantial interface and context work.

Recent research addresses end-to-end automation, control-sequence generation, modular factory skills, and robotic-line control [1]–[4]. Other studies examine agent-based supervision, programmable logic controller (PLC) code generation, context-aware control, resource discovery, shop-floor coordination, digital-twin fault handling, and natural-language machine access [5]–[10], [14]. Recent platforms also ground approved actions in Asset Administration Shell (AAS) models, verify proposals against process and instrumentation diagrams (P&IDs), or organize sensor–manufacturing execution system (MES)–enterprise resource planning (ERP) evidence streams [11]–[13]. Together, these works advance agent reasoning, industrial interfaces, action constraints, and production data integration.

The deployment problem addressed here is the connection between these layers: how to integrate multiple agent engines and roles with distinct industrial scenes, provide the active operating context to each task, and translate a proposal into a bounded, traceable device operation. We present a configuration-driven integration architecture rather than a new control law or optimization algorithm. Its central design choice is to keep agent execution, scene-specific signal and driver mappings, and device-write governance modular while connecting them through named industrial operations.

The contributions are threefold:

1) A scene-integration model binds production context (line–product–recipe–run), semantic acquisition and control nodes, protocol drivers, objectives, and engineering guards. Distinct processes can therefore be represented through configuration and a common operation interface, while protocol-specific access remains in drivers.
2) A collaboration runtime registers harness adapters for different agent engines, assembles current production context at each agent turn, and scopes the available operations to channel policy and resource bindings. Task-based execution and direct-operation entry are connected to the same dispatch design; the paper distinguishes this implemented architecture from the narrower routes exercised in the experiments.
3) A governed write path combines shared dispatch checks with route-specific admission, conditional human approval, post-approval context revalidation, and device-side receipt grading. Owning services retain command receipts separately from later process observations.

Fig. 2 presents the architecture and its module boundaries; Fig. 1 follows one task proposal through operation dispatch, device access, and evidence capture. Evaluation covers five simulator protocol families, a scene-onboarding replay, one agent-mediated Modbus TCP trial on a thermal process twin, six registered-harness tasks, a mock-driver approval-context test, and four separate deterministic missions spanning molding, film, annealing, and wastewater. These checks address different claims and are not pooled as repeated trials. The agent-mediated test establishes one supervised integration chain; the four missions show selected scene-specific operation paths. Neither validates all agent–scene combinations or establishes physical-plant performance.

Section II reviews related work. Section III presents the system architecture and execution flow. Section IV explains agent context, scene integration, and governed device access. Section V reports the protocol, onboarding, agent-task, ap-

proval, and process-mission evidence. Sections VI and VII discuss the findings and conclude the paper.

II. Related Work

Table I summarizes selected studies that connect agent or software-agent functions to industrial services, equipment, or production data and report an implementation or evaluation. The selection locates the integration problem addressed here; it is neither exhaustive nor a matched comparison of system performance.

A. Industrial Agents and Plant Integration

Industrial-agent studies span process supervision, control generation, and equipment-facing assistance. LLM frameworks address end-to-end automation and industrial cyber-physical-system control [1], [2]; related work generates PLC code or combines human guidance with control [5], [6]. Other studies examine resource capabilities, shop-floor coordination, and digital-twin fault handling [7]–[9], as well as natural-language machine access and robotic-line control [4], [10]. Biskupovic et al. are especially close in using agents to supervise industrial systems: their agents select services from a component library using performance and computational-load metrics, monitor online performance, and reconfigure service selection when performance degrades [14]. Taken together, this strand studies agent behavior at the level of control, service choice, or equipment interaction; the question here is how those agents connect to scene-specific signals and governed operations.

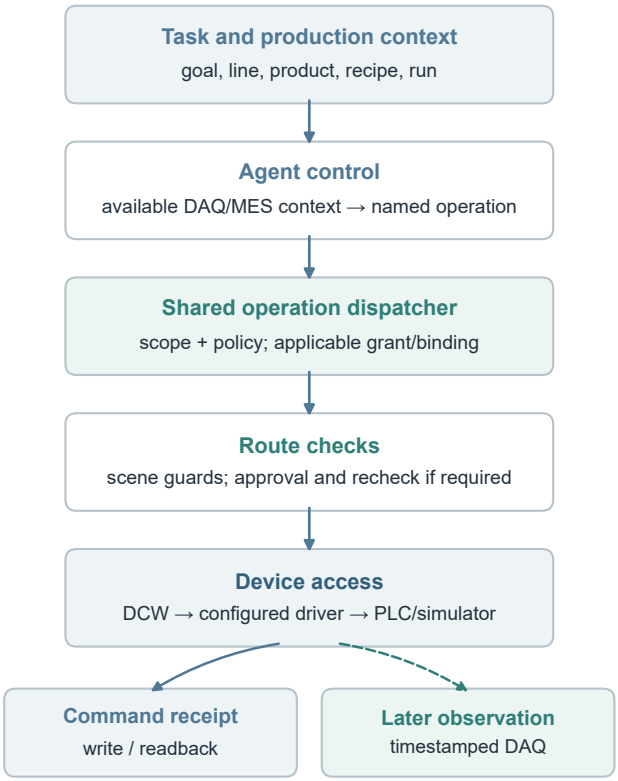


Fig. 1. Task execution path from context to device and observation. The agent uses available acquisition and MES context to propose a named operation. Shared dispatch checks precede route guards and any required approval and context revalidation. Configured drivers provide device access; command receipts and later process observations are retained separately.

Xia et al. connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences [15]. Other nearby designs expose factory-module operations through the Model Context Protocol (MCP) with state-based tool filtering, constrain manufacturing plans, or compose semantic recipes from OPC Unified Architecture (OPC UA) skills [3], [16], [17]. Several recent systems also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [11]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [12]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control on real manufacturing records [13]. These systems address complementary parts of industrial-agent deployment, including process reasoning, service adaptation, asset grounding, action validation, and evidence organization. This paper addresses a related but distinct integration boundary: how to connect replaceable agent harnesses and task roles to different configured production scenes while preserving line–product–recipe–run context, semantic signal mappings, and a governed parameter-write path. Whereas Biskupovic et al. study agent-based service selection and reconfiguration, this architecture centers on scene configuration and a shared operation boundary that resolves configured device nodes and retains command receipts separately from process observations. The contribution is an integration architecture and implementation, not a new process-control law. Table I compares reported mechanisms and evaluation settings only; it does not rank features or performance.

The operation checks complement safety-aware decision support [18] and operate alongside the plant's configured process protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [19]. The platform dispatcher's operation-level scope, policy, grant, and binding checks are distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [20], [21]. Run-to-run methods connect process observations to successive production runs [22]. Human–automation research also formalizes the allocation of information acquisition, analysis, decisions, and actions between people and automation [23]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [24], [25]. These standards

TABLE I
REPORTED OPERATION MECHANISMS AND EVALUATIONS IN SELECTED PLANT-FACING STUDIES. EVALUATION SETTINGS AND OUTCOMES ARE SOURCE-SPECIFIC; THIS TABLE DOES NOT COMPARE PERFORMANCE ACROSS STUDIES.

System	Reported operation or integration mechanism	Reported evaluation
Xia et al. [15]	Cognitive, twin, and physical layers; twin snapshots, event messages, and plan sequences	Robotic automation and process-simulation cases
Biskupovic et al. [14]	Agent-based selection and online reselection of services using performance and computational-load metrics	Reported adaptability and performance improvement in service-oriented industrial cyber-physical systems
Köhle et al. [3]	Injected live state and state-based filtering; MCP-to-OPC UA operations and Message Queuing Telemetry Transport (MQTT) coordination	Nine simulated production challenges
Vieira da Silva et al. [16]	Constrained multi-step plans over MCP manufacturing functions	Laboratory multi-step process
Köhle et al. [17]	Semantic recipes composed from OPC UA skills; virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [11]	AAS-grounded proposals, signed human approval, PLC-validated OPC UA Methods, and immutable audit log	Two laboratory PoCs; single scripted runs
Lin and Chen [13]	Governed sensor–MES–ERP digital thread with SOP-, risk-, and capability-aware action gating	25,275 real records; operational metrics
Schall [12]	P&ID-, SCADA-, and SOP-grounded proposal verification with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
This work	Harness-independent collaboration runtime; scene-bound context and DAQ/DCW mappings; bounded named operations; separate command and observation records	Five-protocol matrix and onboarding; Modbus twin; six harness tasks; approval guard; paired route writes; four single-trajectory missions

and mechanisms describe interfaces for production context, tool interaction, and information flow. Across the selected studies, the reported systems address these capabilities in different application-specific combinations. A remaining gap is an evaluated integration path that binds replaceable agent harnesses to distinct scene configurations, supplies current production context, and preserves a common guarded operation path with separate command and observation records. This paper addresses that gap by supplying scene context to the agent runtime and resolving semantic operations through configured device drivers and execution-time checks. Section V examines selected links in that path under simulator and mock-driver conditions.

III. SYSTEM DESIGN

The architecture connects two configurable sides of an industrial deployment: an agent collaboration runtime and a production-scene configuration. The runtime supplies task context and scoped operations; scene configuration maps process semantics to acquisition and control nodes, drivers, targets, and guards. Named operations connect these sides to shared dispatch and governed device access. The design separates task execution from operation dispatch and keeps command receipts distinct from measured process responses.

A. Design Rationale and Requirements

Five requirements shape the design: scene variability, harness variability, operation authority, outcome observability, and extensibility. Scene configuration binds production records and semantic DAQ/DCW nodes to drivers, objectives, and process guards. Harness adapters isolate agent-engine interfaces from named operations. Shared dispatch checks and route-specific write limits govern authority, while service-owned task, receipt, and observation records preserve outcomes. Registered interfaces allow additional agent engines and device drivers to be integrated without changing the operation contract. Tool visibility alone does not authorize a write; all actuator writes pass through the DCW admission point.

Commissioning associates the line with products, recipes, active-run rules, signal meanings, units, driver mappings, and admissible limits. At runtime, the context builder assembles the active line–product–recipe–run state and relevant process summaries for each agent turn. An operation handler resolves live state again before dispatch, so the prompt context informs a proposal but does not substitute for current authorization or admission checks. Direct operations resolve the current context at invocation without creating a task.

B. Layered Architecture

Fig. 2 summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration runtime, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. A *task route* is an execution path through this runtime; a *harness adapter* connects an agent engine to its task interface. Multiple registered agents can participate through shared tasks and channel context, with each agent's available tools constrained by its binding and channel policy. Direct REST tool calls and the stdio MCP operation bridge bypass

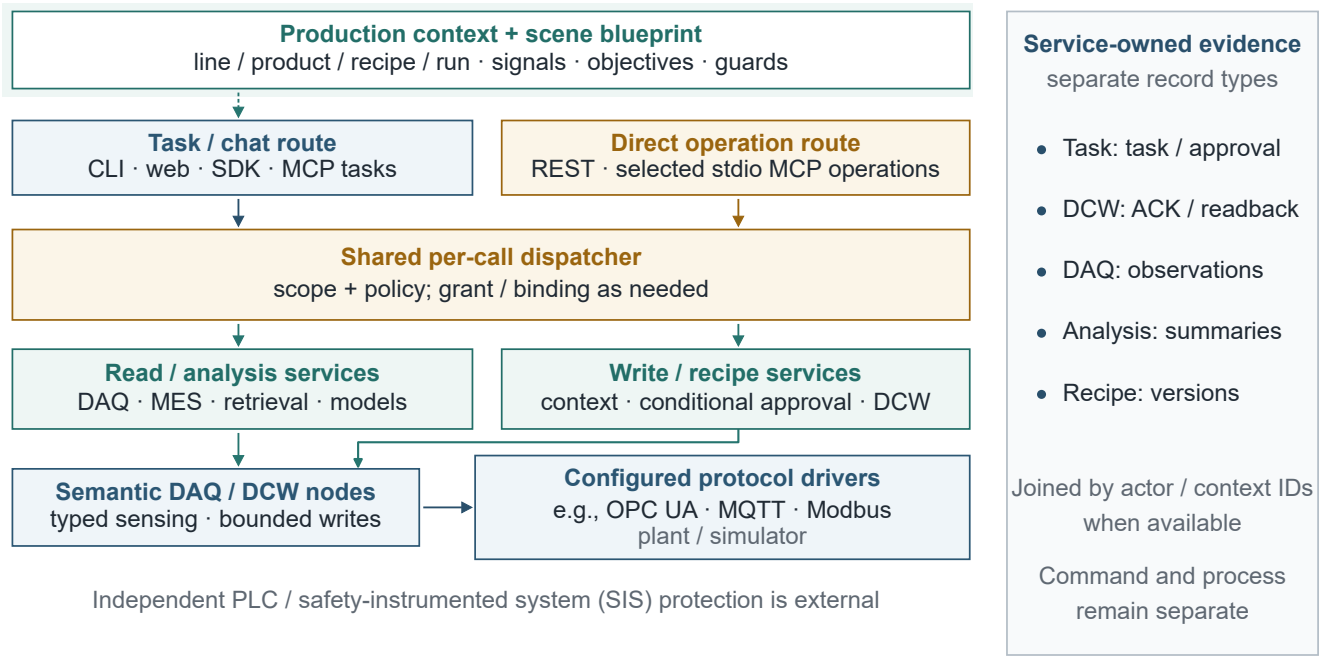


Fig. 2. Architecture and record ownership. Task/chat routes enter the collaboration runtime; direct REST and stdio-MCP calls enter shared dispatch checks without creating a task. One dashed link marks configuration. Write routes apply interval admission and, when configured, approval; recipe dispatch omits the previous-recipe window. The shared dispatcher always checks scope and channel policy, with grant and resource-binding checks applied when required by the operation. Owning services retain distinct records; plant controllers and safety systems retain equipment interlocks.

task execution in that band (the upper-right direct-operation entry in Fig. 2) and enter the shared governance block without creating a task or starting a harness turn; MCP management and task methods remain on their service routes. Direct calls resolve current route state at invocation, without an agent-turn prompt or a task record. The governance block projects the operation catalog and applies the per-call checks. Write handlers apply admission limits and request approval when the operation and its configuration require it. The context block supplies the active line–product–recipe–run tuple to the gate and scene maps, objectives, and guards to the handlers. The dashed link from that block in Fig. 2 marks this configuration relationship; the runtime assembles prompt-time context from the same records. The service band, behind a route-handler interface that selects an adapter, provides acquisition, write control, analysis, retrieval, and optional model services. Field I/O resolves semantic nodes to built-in or plugin drivers for a plant or simulator. Services retain task, command, observation, and configuration records separately, linked to actor and production context.

C. Cross-Module Execution Flow

Fig. 3 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context from current records for each agent turn. For each returned tool call, the shared dispatcher rechecks invocation scope, channel policy, and applicable grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a,c,u) = \bigwedge_{X \in \mathcal{C}(u)} X(a,c,u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, applicable industrial grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S,P,G,B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is included when the resolved channel, target line, and owner/grant policy require it; explicit exemptions omit it. The binding predicate B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. Once these predicates pass, the selected route applies its guards, active-run checks, admission limits, and any required approval. When approval is required, the handler revalidates its context before device dispatch.

For direct REST invocation, the service establishes a and c before applying these checks: an agent token must match the requested agent identifier, or an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record and rechecks scope and policy; applicable grant and binding checks follow. Handlers resolve live production state from request identifiers and owning services.

Fig. 4 gives reference pseudocode summarizing the registered-harness task path. Direct REST tool invocation by-

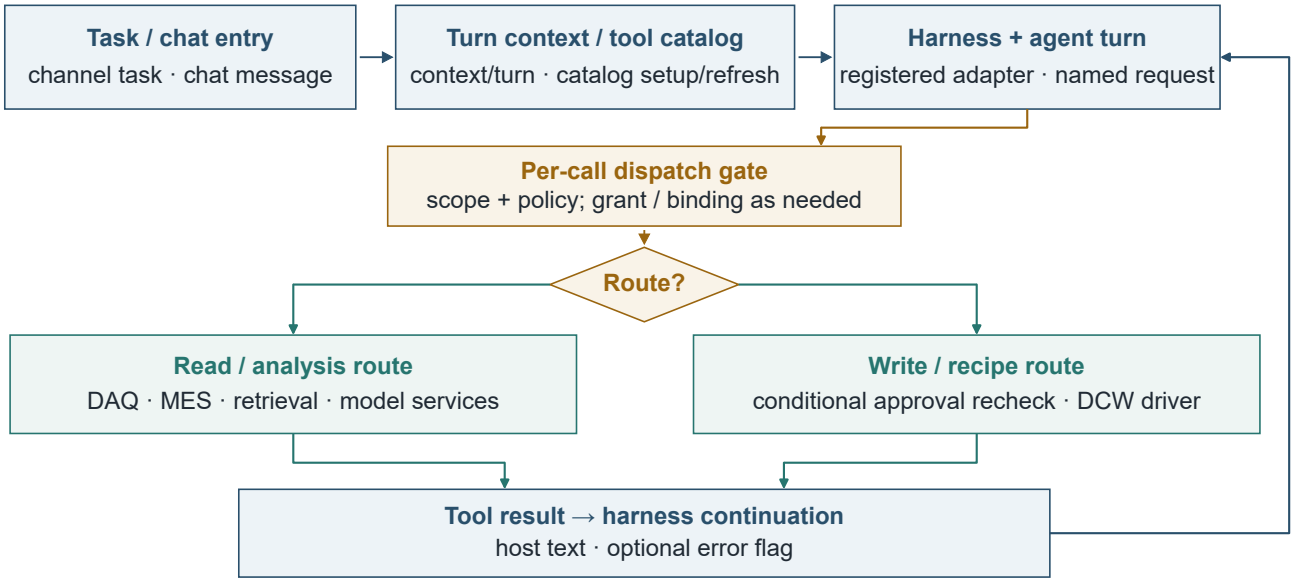


Fig. 3. Registered-harness task flow. The runtime invokes the adapter with a scoped catalog. Scope and channel policy are checked for every call; grant applies to non-exempt industrial operations, and binding applies when resource references are declared. Failed checks return tool errors. The read path retrieves data. The write path applies route guards, conditional approval and its context revalidation, DCW admission, and receipt grading; a later process observation is separate. Results return to the harness. Device-level interlocks remain with the configured plant-control layer.

passes the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

The architecture in Section III assembles task context, dispatches operations, and connects device access to separate records of commands and observations. The following subsections explain the handoffs between these functions.

A. Task, Harness, and Context

Task and chat entry points create or join a workspace channel. The task runtime manages membership, roles, task state, and shared execution context; a harness registry adapts agent engines to a common execution interface. This separation lets agents collaborate through shared tasks and channel resources without coupling their engine-specific turn protocol to a production scene. At each turn, the prompt builder assembles the task, active scene description, line–product–recipe–run state, channel profile, agent binding, and retrieved memory. The resulting context supports process-aware proposals, while the dispatcher independently checks authority and the operation handler resolves current state before a write.

Direct REST and selected stdio-MCP operation calls reach the same dispatcher without a task or harness turn. REST authentication and identity resolution follow Section III-C; each handler reads live route state at invocation. Handlers receive the resolved actor, channel, and operation arguments through the same interface for either entry route; records are keyed by the identifiers supplied in each request. A registered task additionally receives prompt context and a scoped tool catalog. Catalog visibility alone does not grant write authority:

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service

$x_t \leftarrow$ ResolvePromptContext(c, q)
$m \leftarrow$ RetrieveMemory(q)
$p \leftarrow$ AssemblePrompt(q, x_t, m)
$h \leftarrow$ InvokeRegisteredHarness(a, p, T)
for all tool request u returned by h **do**
  $g \leftarrow$ CheckScopeAndChannelPolicy(a, c, u)
  if not g **then**
    EmitToolErrorToHarness(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow$ CheckApplicableGrantBinding(a, u)
    if not v **then**
      EmitToolErrorToHarness(h,u)
      continue
    end if
  end if
  $k \leftarrow$ SelectHandler(u)
  $r \leftarrow$ ExecuteRouteHandler(k, a, c, u) ▷ handler resolves live state and may persist route-specific records
  EmitDeclaredToolResultToHarness(h,r)
end for
return task state and tool results

Fig. 4. Reference pseudocode for the registered-harness task path. Direct operation calls use the shared dispatcher from their own entry point. Each route persists records through its owning service and returns its declared response to its caller; task-route results return to the harness.

the dispatcher checks authority at invocation.

B. Shared Dispatch and Device Admission

The dispatcher is the shared operation boundary. As specified by (1), it checks invocation scope and channel policy for every call, with grant and binding checks applied when required. These checks precede handler selection; handlers revalidate route-specific state at execution time. Scene configuration maps semantic signals and controls to drivers and specifies process objectives and guards. Capability profiles and tool policies narrow available operations, while resource bindings constrain accessible nodes, controls, and recipes. Optional DAQ, MES, analysis, retrieval, optimization, and digital-twin services are invoked through registered handlers. The application supplies its optimization strategy; the framework coordinates context, operation access, and execution.

The target band describes a desired process outcome. It is not the admissible write interval: the latter is computed from engineering, product, recipe, and route-specific limits below. A candidate can therefore satisfy all write constraints yet fail to move the measured process value into its target band.

Actuator writes converge on the shared DCW admission point. For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{L}_n^{\text{base}} \cap \mathcal{L}_n^{\text{prod}}(t) \cap \mathcal{L}_n^{\text{rec}}(t), \qquad (2)$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{L}_n^{\text{base}}$, $\mathcal{L}_n^{\text{prod}}$, and $\mathcal{L}_n^{\text{rec}}$ are the baseline, active-product, and active-recipe limits; the $\mathcal{L}$ limits here are deliberately named apart from the policy and binding predicates of (1). Missing optional limits impose no restriction; an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch validates the replacement recipe against the hard, baseline, and product limits rather than the previous recipe window. Step, interval, and hold-window checks apply as configured for each route. These application-level checks bound commands alongside independent programmable-logic-controller and safety-instrumented protection. They do not replace equipment interlocks or certify the safety of a plant deployment.

C. Approval, Execution, and Evidence

Routes add timing checks and, when required by operation and configuration, approval checks. Outside benchmark mode, online agent and manual node writes enforce a 60 s minimum interval between successful writes. Recipe trials use a per-line cadence (300 s by default), and recipe intervals are anchored to approval. Manual approval binds an actor and proposed operation to candidate values, applicable limits, expiry, and the active line–product–recipe–run tuple. After approval, the handler rechecks authority and compares the current tuple with the approved tuple before entering DCW dispatch. Single-parameter waits expire after 180 s by default; recipe-dispatch approvals use a 30-minute window. A changed recipe definition requires a new `recipe_apply` proposal.

Fig. 5(a) illustrates the four-layer interval in Eq. (2) using rotational-speed limits from a separate archived admission drill. Panel (b) shows the decision and rechecks when approval is required.

A `recipe_trial` illustrates the operation sequence: the handler checks the bound recipe, active run, candidate parameters, and route timing; if required, it requests approval; after revalidation, it submits a bounded batch through the configured driver and returns a per-parameter receipt. The caller can obtain later DAQ observations separately and may then update, apply, or roll back a recipe. A trial neither saves nor applies a recipe version. The driver registry maps semantic controls to configured protocol drivers, keeping protocol details out of agent operation arguments.

For a concrete illustration, consider a simulated coating line whose semantic line-speed control is mapped to a Modbus TCP write node. A request to change the setpoint from 37 to 39 m/min is inside a 28–48 m/min recipe window; the remaining applicable limits and timing checks must also pass. If the configured policy requires approval, the proposal binds the candidate and active production context, and the handler rechecks that context before dispatch. The driver then returns a graded receipt. A later acquisition sample is stored separately and can be associated with the operation when the route provides the necessary context and timestamps. This example illustrates the handoffs, not a recorded human decision or a particular process response; the simulator onboarding replay in Section V used test-only automatic approval.

Receipts and observations answer different questions. A command result is graded as readback-verified, transport-acknowledged, or unverified; a readback mismatch can trigger up to three reads at 2 s intervals. Recipe batches retain per-parameter verification outcomes. A readback-verified result records that the returned value matched the request; a transport acknowledgement alone does not establish that the device applied it, and an unverified result does not establish a matching readback. DAQ stores subsequent timestamped process observations, while recipe history, approvals, and task records remain with their owning services. Identifiers and timestamps can link these records when available, but a later observation alone does not prove that the command caused the measured change. Section V evaluates selected parts of this evidence chain under the stated test conditions.

V. Experimental Evaluation

The evaluation asks whether the architecture connects configured industrial scenes and agent runtimes to protocol-level operations while retaining evidence of authorization, device writes, and process observations. Six evidence sets address separate claims: protocol integration and scene onboarding; one agent-mediated Modbus TCP run on a thermal twin; six registered-harness tasks; a mock-driver approval-context test; a paired entry-path check; and four direct-node missions. The sets use different configurations and endpoints, so they are reported separately rather than combined as one benchmark.

A. Test Design and Evidence Rules

All process runs used local simulators rather than physical equipment. The agent-mediated thermal test used an actual Modbus TCP stack connected to a separate gray-box process simulator. The simulator represented first-order thermal inertia

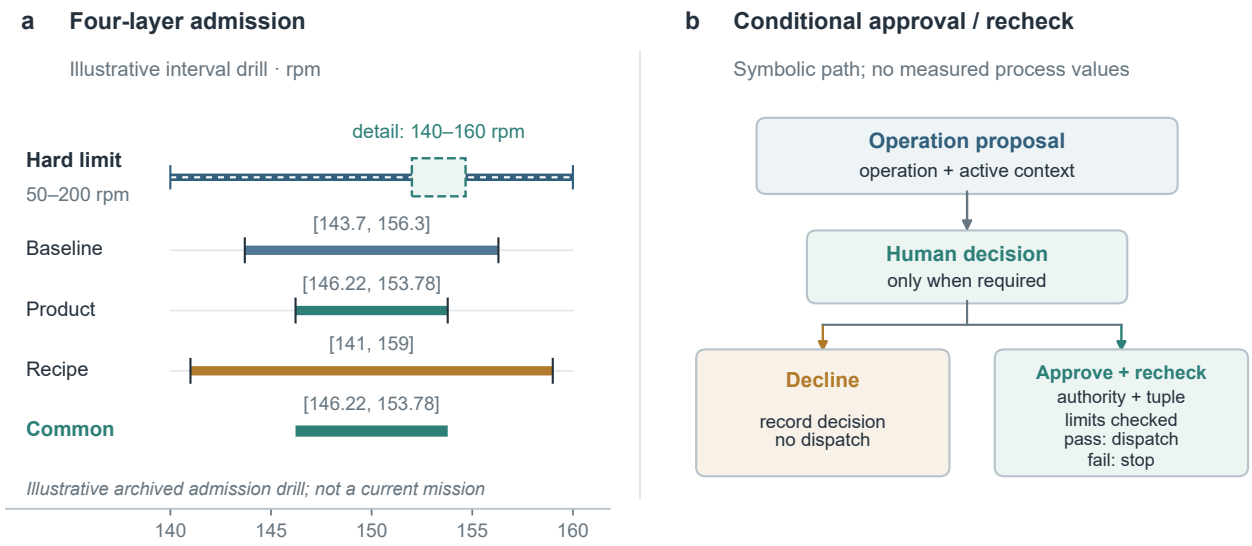


Fig. 5. Admission and approval boundaries. (a) Illustrative rotational-speed limits from a separate archived drill. The hard-limit track spans 50–200 rpm; the lower axis expands its 140–160 rpm segment. Online writes use the intersection in Eq. (2); recipe dispatch omits the previous recipe window. (b) When approval is required, a decline prevents dispatch; approval is followed by authority, context, and limit rechecks.

and delay; it was not calibrated to a production line. An independent Modbus client read the simulated device registers, providing a device-side check separate from the platform's own records. The run used one OMP harness with a local Qwen model in a manual-approval channel. The retained task and tool-call records do not include the model provider's raw request and response, so they support the execution sequence but not an independent audit of the model's reasoning.

The four process missions used deterministic, scenario-specific routines with a fixed seed and the direct-node interface; no language model selected their setpoints. Each mission contributes one trajectory. Its initial observation is a within-trajectory reference, not an independent control. A requested setpoint is an action, not a device readback, and a later process value is an observation rather than proof that a preceding action caused the change. The registered-harness tests used one engine across two simulator scenes; their three tasks per scene shared a simulator instance. Their post-trial samples were explicit driver probes stored through a local SQLite fallback because the isolated online queue and time-series service were unavailable. Full runtime snapshots were not retained for these tests or the four missions.

The protocol matrix, agent-mediated twin, registered-harness tasks, mock-driver test, and process missions are distinct setups. Within the paired entry-path check, the two routes shared each scene's normalized profile and admissible candidate set but used separate route-level entities and different candidates. Each result is interpreted within its own test boundary; no outcomes are pooled.

The paired-entry check used a deterministic local OMP-RPC fixture, with no LLM inference, and direct REST invocation in two simulator scenes; its results are reported separately below.

B. Protocol Integration and Scene Onboarding

The protocol matrix connected simulator endpoints through Modbus TCP, Modbus RTU, OPC UA, MQTT, and HTTP. The platform acquired data from five acquisition nodes and exposed four control nodes. Each protocol produced between 15 and 29 samples that matched simulator-side values within the recorded tolerances. Batch-write checks over four protocol families were confirmed by device-side and platform readback. Values outside an engineering range and single-step limit were rejected without changing the simulated device. An OPC UA disconnection produced a failed connection and write result; after service recovery, a write completed with verified readback. These checks used simulated endpoints with real protocol drivers and do not validate a physical controller or production security configuration.

A separate onboarding replay configured a simulated Modbus TCP device with four control and seven acquisition nodes, plus scalar, vector, table, and image forms from an MES simulator. All seven acquisition nodes yielded samples, four invalid writes were rejected, and four legal writes were confirmed by readback. The replay passed 23 verifier checks; its image data used a local-storage fallback. In a separate BOPET configuration, nine devices exposed 49 signals (30 setpoints and 19 measurements), and four requests addressed three control nodes. These examples show the configuration work needed to map a scene into the common interface; they do not establish zero-effort onboarding.

C. Agent-Mediated Modbus Trial on a Thermal Twin

The end-to-end test configured one acquisition node and three control nodes over Modbus TCP against an isolated extrusion/casting thermal twin. The agent queried the node inventory, active line context, and process data, then submitted a three-zone recipe trial. Its proposed setpoints changed from 198 to 200 °C; after manual approval, the external Modbus

client confirmed all three device registers at 200 °C, matching the platform receipt. The task trace includes an initial acquisition request with an invalid time bound; a later request succeeded. The system therefore completed the governed write path despite a recoverable tool-call error.

A distinct, manually approved follow-on batch changed all three setpoints from 200 to 205 °C. Independent register reads and platform readback confirmed all three values. In the 40 s observation window after this change, the twin's process value rose from 198.02 to 200.73 °C. The platform's corresponding acquisition samples changed from 198.41 to 200.49 °C, and the agent later retrieved 200.49 °C, matching the platform record. The agent-proposed and manually approved changes occurred in one continuous thermal run without an isolated baseline between them. The record therefore cannot separate residual contributions from the 198-to-200 °C trial and the later 200-to-205 °C step. The report records 21 of 21 assertions passing, but this is one simulator run without repeated trials, a matched controller, or a calibrated process model.

Supplementary File S2 presents the event sequence and separates the device receipt from the later process observation.

D. Registered-Harness Tasks

A separate registered harness, configured with `glm-5.3-flash`, executed six sequential tasks across two simulated scenes. Each task supplied a current control read, a process measurement, a target band, and three candidate settings prevalidated against the configured engineering range and step. In the film scene, candidates were 125, 130, and 135 rpm; in annealing, they were 735, 740, and 745 °C. The target bands were not pre-verified. With synthetic administrator approval, each task retained an approved proposal, recipe-trial record, 1/1 verified device receipt, and run-tagged sample. The runner waited 20 s after receipt and readback before probing the driver. Fig. 6 pairs each task's pre-task measurement with its single post-trial probe. All six probes were below their target bands: the three film values were 45.69, 46.69, and 45.69 μm against 48–52 μm; the three annealing values were 679.60, 677.70, and 678.10 °C against 700–760 °C. The records show successful task and write handling, not successful process optimization. Independent model-provider logs were not retained.

E. Approval-Context Guard Test

An independent component test used simulated approval decisions and a mock driver. For recipe application, trial, and rollback, it changed each of the line, product, recipe, and active-run fields after proposal; all stale contexts were refused before mock writes. A further change between authorization revalidation and the final context check was also refused. An unchanged context produced the expected mock writes, while a declined proposal produced none. In a test-only ablation that removed tuple comparison, six product or run changes reached the mock driver and each produced two writes; line and recipe changes remained blocked by other checks. This isolates the tested product/run protection within the component setup, not operator effectiveness or physical safety.

TABLE II
VERIFIED STATE TRANSITIONS IN THE DETERMINISTIC PAIRED ENTRY-PATH CHECK. BOTH ROUTES USED THE SAME NORMALIZED SCENE PROFILE; THEIR RESPECTIVE CANDIDATES WERE DRAWN FROM THE LISTED ADMISSIBLE SET.

Scene	Registered task route	Direct REST route
Cast-film	120→125 rpm; 1/1 receipt; readback 125	120→130 rpm; 1/1 receipt; readback 130
Annealing	730→735°C; 1/1 receipt; readback 735	730→740°C; 1/1 receipt; readback 740

TABLE III
PRIMARY PROCESS VALUES IN FOUR SEPARATE SIMULATED MISSIONS. EACH ROW IS ONE TRAJECTORY; THE FINAL VALUE IS THE MISSION-REPORTED VALUE.

Scene	Target band	Initial → final
Injection	Part weight: 32.15–32.85 g	31.296 → 32.36 g
Film	Thickness: 24.3–25.7 μm	28.03 → 25.30 μm
Annealing	Hardness: 89–101 HV	134.9 → 99.5 HV
Wastewater	Dissolved oxygen: 2.8–4.0 mg/L	0.194 → 3.54 mg/L

F. Paired Entry-Path Check

The paired check exercised the registered task route and direct REST route in two configured scenes: cast-film extrusion over OPC UA and continuous annealing over Modbus TCP. Each scene used the same normalized profile for both entry paths, including its signal mapping, engineering range, 30-unit step limit, target band, and predeclared candidate set. The fixture and REST operation used different admissible candidates within that set. Each route required synthetic administrator approval and produced one recipe-trial record with a verified device acknowledgement. A separate DCW read before and after each operation checked that the register changed to the requested value.

Table II shows the four observed state transitions. All four operations returned a 1/1 verified receipt, and each post-write read matched its requested setpoint. The registered fixture emitted one tool call and one result per scene, but the retained task state was `SUBMITTED`; the test therefore verifies the tool-operation path rather than task completion. It collected no post-write process sample. Because the fixture supplied the calls and the routes used distinct candidates, these results establish neither agent decision quality nor comparative process response.

G. Direct-Node Missions Across Four Process Scenes

Table III reports the primary process value from each of the four deterministic missions. Each mission used a distinct simulated process and one trajectory; the agent-mediated test and harness tasks above used different setups and are neither controls nor repetitions of these missions.

Each mission involved multiple parameter changes, so the single trajectories do not isolate individual action effects. Supplementary File S4 records the reported sequences and secondary measurements. It also identifies the simulator-defined wastewater cost index as a proxy rather than a measured operating cost. Final annealing tensile-strength and surface-defect readings, and the complete final wastewater effluent vector, were not archived.

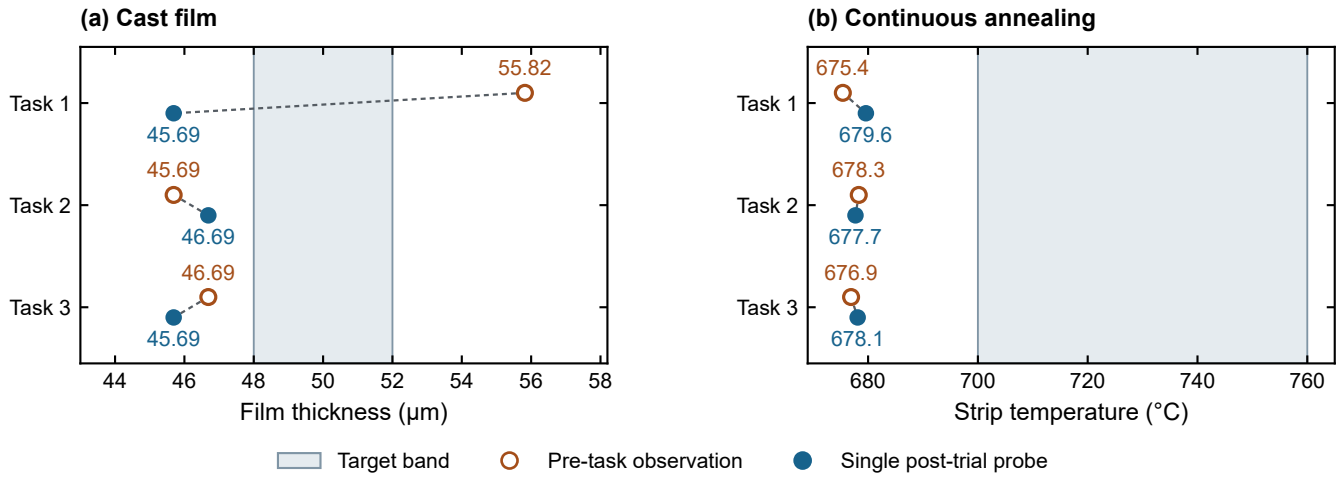


Fig. 6. Pre-task observations and single post-trial probes for six registered-harness tasks in two simulated scenes. Shading denotes the target band; task numbers indicate order within each scene. Dashed connectors pair observations within a task, and vertical offsets separate markers without changing measured values. The connectors are not a time series. The runner waited 20 s after receipt and readback before each probe. Tasks in each scene shared one simulator instance; synthetic approval and a local SQLite fallback were used.

VI. Discussion

A. Interpretation

The protocol matrix and onboarding replay show that configured drivers and scene mappings can expose the tested heterogeneous simulator endpoints. The agent-mediated thermal-twin run provides the fullest task trace: after context retrieval and manual approval, independent register reads confirmed the three proposed setpoints, and the agent later retrieved a DAQ sample matching the platform record. A subsequent 200-to-205 °C step preceded a 2.71 °C rise over 40 s. Both recipe changes occurred in one continuous run, so their effects cannot be separated; the gray-box twin was not calibrated to a production process.

The paired-entry check addresses software routing. Its deterministic fixture and direct REST call each completed approved writes in two scene profiles; all four platform acknowledgements and separate device reads matched their requested values. The routes used separate entities, so the result is a profile-matched integration check rather than a same-context route comparison. The fixture made no agent decision, the task remained submitted, and no process samples were collected. The six registered-harness probes remained below their target bands, whereas each separate direct-node mission reported one endpoint within its band. These are different tests and do not form a performance comparison or establish a parameter's causal effect.

The architecture provides extension points for additional harnesses and production scenes, but the tests do not validate role-separated multi-agent control, all agent–scene combinations, commissioning effort, or a reuse benefit. The paired fixture is not evidence of agent behavior.

B. Scope and Limitations

All process evidence comes from local simulation. Each of the four direct-node missions is one deterministic trajectory without a matched controller or independent replication. Full runtime snapshots and continuous device traces were not retained for every experiment, and provider request/response logs are unavailable for the agent-mediated run. The results therefore support selected execution events and reported endpoints, not complete replay, repeatability, or process-control benefit.

The harness tasks used synthetic approval; the thermal trial used one manual approval configuration. The mock-driver test examines stale-context refusal, not operator decision quality or review effectiveness [26], [27]. Final annealing strength/defect readings and the complete wastewater effluent vector were not archived. The software checks do not constitute a safety or security assessment; deployment still requires site-specific protocol security, device protection, and independent PLC or safety-instrumented interlocks [28]–[31].

Further evaluation should join a real role-separated agent team to a shared scene configuration and the complete context–approval–write–readback– observation chain, then repeat matched workflows and process missions across initial conditions under operator oversight. Until then, the evidence supports the tested software integrations, not field performance, lower commissioning cost, or autonomous operation.

VII. Conclusion

This paper presented a configuration-driven architecture that connects agent runtimes and per-turn production context to scene-specific acquisition and control mappings. Named operations pass through shared dispatch and route-specific write limits; applicable approvals trigger context revalidation, and command receipts remain separate from process observations.

Evaluation covered five simulator protocol families, onboarding, one agent-mediated Modbus-twin run, six registered-harness tasks, a mock-driver guard test, paired task/direct-REST writes in two scene profiles, and four single-trajectory process missions. The paired fixture verified 4/4 writes with

matching device readbacks; the six task probes remained below target, while each mission reported one endpoint within its band. The evidence supports selected simulator integration paths, but not multi-agent performance, repeatable process benefit, human-review effectiveness, or physical-plant performance.

Data Availability: Historical Supplementary File S1 does not contain the current four-mission source trajectories. Supplementary Files S2–S4 summarize the thermal-twin run, paired-entry check, and process missions; they are not full replay packages. A sanitized run record and local service logs for the paired-entry check accompany the manuscript source and figures. Supplementary File S4 contains reported mission values, but the raw four-mission trajectories are unavailable. No physical-plant data are included.

REFERENCES

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[12] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[13] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[14] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[15] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[16] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[17] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[18] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[19] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[20] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[21] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[22] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[23] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[24] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[25] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[26] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[27] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[29] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[30] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[31] ——, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.